Spring AOP Pointcut Expressions

1. What Is a Pointcut Expression?

An advice defines **what additional behaviour should run**, such as logging, security, auditing or performance tracking.

A pointcut expression defines **where that advice should run**.

```
@Before("execution(* getStudent(..))")
public void logBeforeMethod() {
    System.out.println("Method is about to execute");
}
```

Here:

- `@Before` defines when the advice runs.
- `execution(* getStudent(..))` selects the target methods.
- `logBeforeMethod()` contains the additional behaviour.

> A pointcut expression does not execute a method. It decides whether a method execution should be intercepted.

---

2. The `execution()` Designator

`execution()` is the most commonly used pointcut designator in Spring AOP. It selects methods from their signatures.

It can match a method using its:

- Access modifier
- Return type
- Declaring class

- Method name
- Parameters
- Declared exceptions

Complete syntax:

```
execution(
    modifiers-pattern?
    return-type-pattern
    declaring-type-pattern?
    method-name-pattern(parameter-pattern)
    throws-pattern?
)
```

The parts marked with `?` are optional. The essential parts are:

```
Return type + method name + parameters
```

The broadest commonly used expression is:

```
execution(* *(..))
```

```
*     → any return type
*     → any method name
(..)  → zero or more parameters
```

2.1 Exact Method Matching

Consider:

```
package com.coderarmy.student.service;

public class StudentService {
```

```
    public Student getStudent(Long id) {
        return null;
    }
}
```

An exact pointcut is:

```
execution(public com.coderarmy.student.model.Student com.coderarmy.student.service.StudentService.getStudent(java.lang.Long))
```

Part	Meaning
`public`	The method must be public
`com.coderarmy.student.model.Student`	The return type must be `Student`
`com.coderarmy.student.service.StudentService`	The method must belong to `StudentService`
`getStudent`	The method name must be `getStudent`
`java.lang.Long`	It must accept exactly one `Long` argument

This expression matches only that exact signature. Real applications normally use broader expressions so that one pointcut can cover multiple related methods.

---

3. Wildcards: `*` and `..`

The meaning of a wildcard depends on where it appears.

3.1 The `*` Wildcard

- matches anything in one position or one naming segment.

Any return type

```
execution(* StudentService.getStudent(..))
```

The method may return `Student`, `String`, `void`, `int` or any other type.

Any method name

```
execution(* *(..))
```

Method names beginning with `get`

```
execution(* get*(..))
```

Matches:

```
getStudent()
getAllStudents()
getStudentById()
getStudentCount()
```

Method names ending with `Student`

```
execution(* *Student(..))
```

Matches:

```
getStudent()
createStudent()
deleteStudent()
```

Method names containing `Student`

```
execution(* *Student*(..))
```

Matches:

```
getStudent()
getStudentById()
createStudentAccount()
updateStudentDetails()
```

Exactly one parameter of any type

```
execution(* *(*))
```

Matches:

```
getStudent(Long id)
saveStudent(Student student)
deleteStudent(String email)
```

It does not match methods with zero or multiple parameters.

```
(*)  → exactly one parameter of any type
(..) → zero or more parameters of any type
```

3.2 The `..` Wildcard

`..` represents zero or more elements.

Inside a parameter list

```
execution(* *(..))
```

It matches methods with any number of parameters, including none:

```
getAllStudents()
getStudent(Long id)
```

```
updateStudent(Long id, Student student)
search(String name, Integer age, String city)
```

Inside a package pattern

```
com.coderarmy.student.service..*
```

It matches the package and all its subpackages:

```
com.coderarmy.student.service.StudentService
com.coderarmy.student.service.impl.StudentServiceImpl
com.coderarmy.student.service.internal.StudentValidator
```

By comparison:

```
com.coderarmy.student.service.*
```

matches only classes placed directly inside `service`. It does not include `service.impl` or other subpackages.

```
*   → one package or type-name segment
..  → zero or more package segments
```

---

4. Common `execution()` Patterns

Consider the following service:

```
public class StudentService {

    public Student getStudent(Long id) {
        return null;
    }

    public List<Student> getAllStudents() {
```

```
        return null;
    }

    public Student createStudent(Student student) {
        return null;
    }

    public Student updateStudent(Long id, Student student) {
        return null;
    }

    public void deleteStudent(Long id) {
    }
}
```

Match a method name everywhere

```
execution(* getStudent(..))
```

This can match any interceptable method named `getStudent`, including methods in controllers, services or other classes.

A safer expression is:

```
execution(* com.coderarmy.student.service.StudentService.getS
tudent(..))
```

Match every method

```
execution(* *(..))
```

This is extremely broad and should normally be restricted by package, class, bean name or annotation.

Match public methods

```
execution(public * *(..))
```

The modifier is optional. Public methods are the safest and most common targets in proxy-based Spring AOP. Private methods cannot be intercepted through normal proxy overriding.

Match all public methods in the service layer

```
execution(public * com.coderarmy.student.service..*.*(..))
```

Part	Meaning
`public`	Public methods only
first `*`	Any return type
`com.coderarmy.student.service..*`	Any class in the package or its subpackages
second `*`	Any method name
`(..)`	Any number and type of arguments

Quick examples

Requirement	Expression
Any method	`execution(* *(..))`
Any method named `getStudent`	`execution(* getStudent(..))`
Methods beginning with `get`	`execution(* get*(..))`
Any method in `StudentService`	`execution(* com.coderarmy.student.service.StudentService.*(..))`
Methods in the service package only	`execution(* com.coderarmy.student.service.*.*(..))`
Methods in the service package and subpackages	`execution(* com.coderarmy.student.service..*.*(..))`
Methods with no parameters	`execution(* *())`

Requirement	Expression
Methods with exactly one parameter	`execution(* *(*))`
Methods accepting exactly one `Long`	`execution(* *(java.lang.Long))`
Methods whose first parameter is `Long`	`execution(* *(java.lang.Long, ..))`

5. The `within()` Designator

`execution()` focuses on a method signature. `within()` focuses on the class or package where the method is declared.

```
execution() → Which method signature should match?
within()    → Inside which class or package should methods match?
```

Exact class

```
within(com.coderarmy.student.service.StudentService)
```

Classes directly inside a package

```
within(com.coderarmy.student.service.*)
```

A package and all its subpackages

```
within(com.coderarmy.student.service..*)
```

Practical example:

```
@Before("within(com.coderarmy.student.service..*)")
public void logServiceMethod() {
```

```
    System.out.println("Service method called");
}
```


6. `execution()` vs `within()`

These expressions may produce similar results for a basic service-layer requirement:

```
execution(* com.coderarmy.student.service..*.*(..))
```

```
within(com.coderarmy.student.service..*)
```

The difference is expressive power.

`execution()` can filter by modifier, return type, method name and parameters. `within()` only restricts the declaring class or package.

They can also be combined:

```
within(com.coderarmy.student.service..*)
&& execution(public * get*(..))
```

Meaning:

> Match public methods beginning with `get`, but only inside the service layer.

7. The `@annotation()` Designator

`@annotation()` matches methods that carry a specified annotation.

```
@Transactional
public void transferMoney() {
}
```

Pointcut:

```
@annotation(org.springframework.transaction.annotation.Transa
ctional)
```

It means:

> Match a method execution when the method has the specified annotation.

`@annotation()` is used for method-level annotations. Class-level annotation matching is handled by `@within()` and `@target()`.

Why annotation-based matching is useful

A package-based pointcut applies behaviour throughout an architectural region. An annotation-based pointcut applies behaviour only to methods that explicitly opt in.

Common use cases include:

- Auditing selected operations
- Measuring execution time
- Recording sensitive actions
- Custom authorization
- Idempotency protection

Example with a custom annotation:

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface TrackExecutionTime {
}
```

```
@TrackExecutionTime
public Student createStudent(Student student) {
    return student;
}
```

```
@Around("@annotation(com.coderarmy.aop.TrackExecutionTime)")
public Object measureExecutionTime(ProceedingJoinPoint joinPo
int)
        throws Throwable {

    long start = System.currentTimeMillis();
    Object result = joinPoint.proceed();
    long end = System.currentTimeMillis();

    System.out.println("Execution time: " + (end - start) + "
ms");
    return result;
}
```

8. The `bean()` Designator

`bean()` matches methods using the Spring bean name rather than the Java class name.

```
@Service
public class StudentService {
}
```

The default bean name is normally:

```
studentService
```

Pointcut:

```
bean(studentService)
```

Example:

```
@Before("bean(studentService)")
public void beforeStudentService() {
    System.out.println("Student service method called");
}
```

Wildcard matching is also supported:

```
bean(*Service)
```

This can match bean names such as:

```
studentService
teacherService
paymentService
emailService
```

> `bean()` is specific to Spring AOP and works with Spring-managed bean names.

---

9. Combining Pointcut Expressions

Pointcut expressions can be combined using boolean operators:

```
&&  // AND
||  // OR
!   // NOT
```

AND: `&&`

Both expressions must match.

```
within(com.coderarmy.student.service..*)
&& execution(public * *(..))
```

Matches public methods inside the service layer.

OR: `||`

At least one expression must match.

```
within(com.coderarmy.student.service..*)
|| within(com.coderarmy.student.controller..*)
```

Matches methods in either the service layer or the controller layer.

NOT: `!`

The negated expression must not match.

```
within(com.coderarmy.student.service..*)
&& !execution(* getHealth(..))
```

Matches service methods except methods named `getHealth`.

Another example:

```
within(com.coderarmy.student..*)
&& !within(com.coderarmy.student.repository..*)
```

Matches application methods except repository methods.

Use parentheses to keep complex expressions readable:

```
(
    within(com.coderarmy.student.service..*)
    || within(com.coderarmy.student.controller..*)
)
&& execution(public * *(..))
```

---

10. Named Pointcuts with `@Pointcut`

Long expressions are often reused across multiple advice methods. Repeating them creates duplication and makes maintenance harder.

A named pointcut assigns a reusable name to an expression.

```
@Pointcut("within(com.coderarmy.student.service..*)")
public void serviceLayer() {
}
```

The method body remains empty. Its name represents the pointcut expression.

Reuse it in an advice:

```
@Before("serviceLayer()")
public void beforeServiceMethod() {
    System.out.println("Before service method");
}
```

`serviceLayer()` is not called as a normal Java method. Spring resolves it as a pointcut declaration.

---

11. Centralizing Application Pointcuts

When several aspects share the same layer definitions, keep them in one class.

```
package com.coderarmy.student.aop;

import org.aspectj.lang.annotation.Pointcut;

public class ApplicationPointcuts {

    @Pointcut("within(com.coderarmy.student.controller..*)")
    public void controllerLayer() {
    }

    @Pointcut("within(com.coderarmy.student.service..*)")
    public void serviceLayer() {
    }
```

```
    @Pointcut("within(com.coderarmy.student.repository..*)")
    public void repositoryLayer() {
    }

    @Pointcut("execution(public * *(..))")
    public void publicMethod() {
    }

    @Pointcut("serviceLayer() && publicMethod()")
    public void publicServiceMethod() {
    }
}
```

Reference it from another aspect using its fully qualified name:

```
@Aspect
@Component
public class LoggingAspect {

    @Before(
        "com.coderarmy.student.aop.ApplicationPointcuts.serviceLayer()"
    )
    public void logBeforeServiceMethod() {
        System.out.println("Service method called");
    }
}
```

This creates one architectural definition of each layer. If a package changes, the expression needs to be updated in only one place.

---

12. Advanced Pointcut Designators

12.1 `@within()`

```
@within(com.coderarmy.aop.Audited)
```

Matches methods declared inside types carrying the specified annotation.

```
@Audited
@Service
public class PaymentService {

    public void processPayment() {
    }
}
```

12.2 `@target()`

```
@target(com.coderarmy.aop.Audited)
```

Matches when the runtime target object's class carries the specified annotation.

In an ordinary class, `@within()` and `@target()` may appear to behave the same:

```
@TrackExecution
@Service
public class StudentService {

    public void addStudent() {
    }
}
```

The difference becomes clearer with inheritance.

```
public class BaseService {

    public void save() {
        System.out.println("Saving");
```

```
    }
}
```

```
@TrackExecution
@Service
public class StudentService extends BaseService {
}
```

When `studentService.save()` executes:

- `@target(TrackExecution)` matches because the runtime target object is a `StudentService`.
- `@within(TrackExecution)` does not match because `save()` was declared inside the unannotated `BaseService`.

```
@within → checks the type where the method is declared
@target → checks the runtime target object's class
```

---

13. `args()` and `@args()`

`args()`

`args()` checks runtime argument types.

```
@Before("args(com.coderarmy.student.model.Student)")
public void beforeStudentArgument() {
    System.out.println("Student argument received");
}
```

It matches when the runtime argument is compatible with `Student`.

`@args()`

`@args()` checks annotations on the runtime classes of the arguments.

```java
@TrackData
public class Student {

    private String name;
}
```

```java
public void saveStudent(Student student) {
}
```

```java
@Before("@args(com.coderarmy.annotation.TrackData)")
public void beforeMethod() {
    System.out.println("Annotated argument received");
}
```

When `new Student()` is passed, Spring checks the runtime class of that object. Because `Student` carries `@TrackData`, the pointcut matches.

```
args(Type)             → runtime argument type
@args(AnnotationType)  → annotation on the runtime argument type
```

`@args()` does not check whether the method parameter itself is annotated.

---

14. `this()` and `target()`

Spring AOP normally places a proxy in front of the actual Spring bean.

```
Caller → Spring proxy → Actual target bean
```

In pointcut terminology:

```
Proxy object       → this
Actual bean object → target
```

Therefore:

```
this(Type)   → checks the Spring proxy type
target(Type) → checks the actual target bean type
```

Consider:

```
public interface StudentOperations {

    void addStudent();
}
```

```
@Service
public class StudentService implements StudentOperations {

    @Override
    public void addStudent() {
        System.out.println("Student added");
    }
}
```

target(StudentService)

```
@Before("target(com.coderarmy.student.service.StudentServic
e)")
```

Matches because the actual bean is a `StudentService`.

this(StudentOperations)

```
@Before("this(com.coderarmy.student.service.StudentOperation
s)")
```

With a JDK dynamic proxy, this matches because the proxy implements `StudentOperations`.

15. JDK Proxy vs CGLIB Proxy

JDK dynamic proxy

A JDK proxy exposes the implemented interfaces.

```
Proxy type  → StudentOperations
Target type → StudentService
```

Therefore:

```
this(StudentOperations)  → matches
target(StudentService)   → matches
this(StudentService)     → normally does not match
```

The proxy implements `StudentOperations`, but it is not a subclass of the concrete `StudentService` class.

CGLIB proxy

A CGLIB proxy subclasses the target class.

```
StudentService
      ↑
StudentService$$SpringCGLIB$$...
```

Therefore, both can match:

```
this(com.coderarmy.student.service.StudentService)
```

```
target(com.coderarmy.student.service.StudentService)
```

The difference between `this()` and `target()` is most visible with JDK interface-based proxies.

---

16. `target()` vs `@target()`

These designators check different things.

```
target(com.coderarmy.student.service.StudentService)
```

Checks the Java type of the actual target object.

```
@target(com.coderarmy.aop.TrackExecution)
```

Checks whether the actual target object's class carries `@TrackExecution`.

```
this(Type)          → proxy type
target(Type)        → real bean type
@target(Annotation) → annotation on the real bean's class
```

Memory diagram:

```
Caller → [proxy] → [actual bean]
           this       target
```

---

17. Configuring the Proxy Type

Spring Boot

Spring Boot normally uses class-based proxying for AOP auto-configuration.

```
spring.aop.proxy-target-class=true
```

To prefer JDK interface-based proxies:

```
spring.aop.proxy-target-class=false
```

JDK proxies require interfaces. If a class does not implement an interface, Spring uses a class-based proxy instead.

Plain Spring Framework

Class-based proxies:

```
@Configuration
@EnableAspectJAutoProxy(proxyTargetClass = true)
public class AopConfig {
}
```

Interface-based proxies:

```
@Configuration
@EnableAspectJAutoProxy(proxyTargetClass = false)
public class AopConfig {
}
```

The default value of `proxyTargetClass` in `@EnableAspectJAutoProxy` is `false`.

In a normal Spring Boot application, the application property is generally preferred because Boot already provides AOP auto-configuration.

---

18. Quick Reference

Designator	What it checks	Example
`execution()`	Method signature	`execution(* service..*.*(..))`
`within()`	Declaring class or package	`within(service..*)`
`@annotation()`	Annotation on the method	`@annotation(TrackExecutionTime)`
`bean()`	Spring bean name	`bean(*Service)`
`@within()`	Annotation on the declaring type	`@within(Audited)`

Designator	What it checks	Example
`@target()`	Annotation on the runtime target class	`@target(Audited)`
`args()`	Runtime argument types	`args(Student)`
`@args()`	Annotations on runtime argument types	`@args(TrackData)`
`this()`	Spring proxy type	`this(StudentOperations)`
`target()`	Actual target bean type	`target(StudentService)`

19. Practical Guidelines

1. Prefer pointcuts that represent clear architectural boundaries.

```
within(com.coderarmy.student.service..*)
```

is safer than:

```
execution(* *(..))
```

1. Use `execution()` when matching depends on method name, return type, modifier or parameters.
2. Use `within()` when the requirement applies to a complete class, package or application layer.
3. Use `@annotation()` when selected methods should explicitly opt into the aspect behaviour.
4. Use named pointcuts instead of repeating long expressions.
5. Keep pointcuts narrow enough to understand and test. Overly broad expressions can produce excessive logs, unnecessary overhead and unexpected behaviour.
6. Remember that Spring AOP works through Spring-managed proxies:

```
External caller → Spring proxy → target method
```

A direct internal call from one method to another method in the same object normally bypasses the proxy and does not trigger the advice.

---

20. Final Mental Model

A pointcut answers:

> Which method executions should receive the aspect behaviour?

```
Method signature      → execution()
Class or package      → within()
Method annotation     → @annotation()
Spring bean name      → bean()
Runtime argument type → args()
Proxy type            → this()
Target bean type      → target()
```

For most Spring applications, the most important designators are:

```
execution()
within()
@annotation()
bean()
```

The advanced designators become useful when inheritance, runtime argument types or proxy implementation details matter.